

model_development

February 1, 2026

```
[1]: %pip install pyvista
import time
import torch
import pyvista as pv
import matplotlib.pyplot as plt
import yaml
import numpy as np
from typing import OrderedDict
```

Requirement already satisfied: pyvista in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (0.46.4)
Requirement already satisfied: pooch in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (1.8.2)
Requirement already satisfied: pillow in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (12.0.0)
Requirement already satisfied: vtk!=9.4.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (9.5.2)
Requirement already satisfied: scooby>=0.5.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (0.11.0)
Requirement already satisfied: matplotlib>=3.0.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (3.10.7)
Requirement already satisfied: typing-extensions>=4.10 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (4.15.0)
Requirement already satisfied: numpy>=1.21.0 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pyvista) (2.2.6)
Requirement already satisfied: fonttools>=4.22.0 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (4.61.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (25.0)
Requirement already satisfied: python-dateutil>=2.7 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (2.9.0.post0)
Requirement already satisfied: cycler>=0.10 in /home/moritz/numsim-

project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (0.12.1)

Requirement already satisfied: contourpy>=1.0.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (1.3.2)

Requirement already satisfied: pyparsing>=3 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from matplotlib>=3.0.1->pyvista) (3.2.5)

Requirement already satisfied: requests>=2.19.0 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pooch->pyvista) (2.32.5)

Requirement already satisfied: platformdirs>=2.5.0 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from pooch->pyvista) (4.5.1)

Requirement already satisfied: six>=1.5 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib>=3.0.1->pyvista) (1.17.0)

Requirement already satisfied: charset_normalizer<4,>=2 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from requests>=2.19.0->pooch->pyvista) (3.4.4)

Requirement already satisfied: certifi>=2017.4.17 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from requests>=2.19.0->pooch->pyvista) (2025.11.12)

Requirement already satisfied: urllib3<3,>=1.21.1 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from requests>=2.19.0->pooch->pyvista) (2.6.1)

Requirement already satisfied: idna<4,>=2.5 in /home/moritz/numsim-project/.venv/lib/python3.10/site-packages (from requests>=2.19.0->pooch->pyvista) (3.11)

Note: you may need to restart the kernel to use updated packages.

1 helper functions

```
[2]: # from mymodel import init_my_model #for kaggle, the initmymodel def is in a
      ↪code box

def load_data(dataset_path):
    """
    Load dataset from a given path of the format .pt
    It has two tensors: input data and label data

    :param dataset_path: Path to the dataset file
    :return: Loaded dataset
    """
    print(f"Loading dataset from {dataset_path}")
    # Use safe globals to allow loading TensorDataset
    with torch.serialization.safe_globals([torch.utils.data.dataset.
      ↪TensorDataset]):
```

```

        dataset = torch.load(dataset_path)
        print(
            f"shape of loaded dataset: Input: {dataset.tensors[0].shape} and_
↪labels: {dataset.tensors[1].shape}"
        )
        return dataset

def save_params(model, filepath):
    """
    Save the params of the model to a given filepath

    :param model: Model to be saved
    :param filepath: Path to save the model
    """
    print(f"Saving model to {filepath}")
    torch.save(model.state_dict(), filepath)

def load_params(model, filepath):
    """
    Load the params of the model from a given filepath

    :param model: Model to load the params into
    :param filepath: Path to load the model from
    """
    print(f>Loading params from {filepath}")
    model.load_state_dict(torch.load(filepath))
    return model

def plot_loss(train_losses, val_losses):
    plt.figure(figsize=(10, 5))
    plt.semilogy(train_losses, label="Train Loss")
    plt.semilogy(val_losses, label="Validation Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss MSE")
    plt.title("Training and Validation Loss over Epochs")
    plt.legend()
    plt.show()

def evaluate_model(model, criterion, device, test_dataset):
    model.eval()
    with torch.no_grad():
        y_pred_all = model(test_dataset.tensors[0].to(device))
        #test_loss = criterion(y_pred_all, test_dataset.tensors[1]).item()

```

```

        test_loss = criterion(y_pred_all, test_dataset.tensors[1].to(device)).
        ↪item()

    print(f"Test Loss: {test_loss}")
    return test_loss, y_pred_all

def plot_test_results(test_dataset, y_pred_all, num_samples: int = 3):
    test_inputs = test_dataset.tensors[0].cpu()
    test_labels = test_dataset.tensors[1].cpu()
    test_outputs = y_pred_all.cpu()

    for i in range(num_samples):
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 3, 1)
        plt.title("Input")
        plt.imshow(test_inputs[i, 0, :, :], cmap="viridis")
        plt.colorbar()

        plt.subplot(1, 3, 2)
        plt.title("Ground Truth")
        plt.imshow(test_labels[i, 0, :, :], cmap="viridis")
        plt.colorbar()

        plt.subplot(1, 3, 3)
        plt.title("Prediction")
        plt.imshow(test_outputs[i, 0, :, :], cmap="viridis")
        plt.colorbar()

        plt.show()

def generate_data(folder_path, velocities_range, num_samples):
    input_all = []
    label_all = []
    velocities = np.linspace(
        velocities_range[0], velocities_range[1], num=num_samples
    )

    # Read all files
    for i in range(num_samples):
        # grid = pv.read(f"{folder_path}output_{i}.vti")
        grid = pv.read(f"{folder_path}output_{i}.vti") #Kaggle
        # grid = pv.read(f"{folder_path}output_{str(i).zfill(4)}.vti")

        u = np.array(grid.point_data["velocity"][:, 0]) # u component

```

```

v = np.array(grid.point_data["velocity"][:, 1]) # v component

dims = grid.dimensions # (nx, ny, nz) (21, 21, 1)
u = u.reshape(dims)
v = v.reshape(dims)

uv_combined = np.concatenate([u, v], axis=-1) # (21, 21, 2)

input = np.zeros(dims)
input[-1, 1:dims[1] - 1, 0] = velocities[i] # (21, 21, 1)

input_all.append(input)
label_all.append(uv_combined)

input_all = np.array(input_all)
input_all = np.transpose(input_all, (0, 3, 1, 2)) # Now shape is
↪(num_files, channels=1, ny, nx)
label_all = np.array(label_all)
label_all = np.transpose(label_all, (0, 3, 1, 2)) # Now shape is
↪(num_files, channels=2, ny, nx)

print(f"Generated {len(label_all)} samples of data.")
print(f"Each sample has shape: {label_all[0].shape} for labels and
↪{input_all[0].shape} for inputs.")
print(f"Label data shape: {label_all.shape}")
print(f"Input data shape: {input_all.shape}")

return label_all, input_all

def plot_sample_data(label_all, input_all, sample_index):
    sample = label_all[sample_index] # shape (channels=2, ny, nx)
    sample_input = input_all[sample_index] # shape (channels=1, ny, nx)
    u_component = sample[0, :, :]
    v_component = sample[1, :, :]

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 3, 1)
    plt.title("U Component")
    plt.imshow(u_component, cmap="viridis")
    plt.colorbar()

    plt.subplot(1, 3, 2)
    plt.title("V Component")
    plt.imshow(v_component, cmap="viridis")
    plt.colorbar()

```

```

plt.subplot(1, 3, 3)
plt.title("Input U Component")
plt.imshow(sample_input[0, :, :], cmap="viridis")
plt.colorbar()

plt.show()

def normalize_data(label_data, input_data, yaml_filepath):
    u_min = label_data[:, 0, :, :].min()
    u_max = label_data[:, 0, :, :].max()
    v_min = label_data[:, 1, :, :].min()
    v_max = label_data[:, 1, :, :].max()
    u_input_min = input_data[:, 0, :, :].min()
    u_input_max = input_data[:, 0, :, :].max()
    print(f"Input U component original range: [{u_input_min}, {u_input_max}]")
    print(f"U component original range: [{u_min}, {u_max}]")
    print(f"V component original range: [{v_min}, {v_max}]")

    # scale labels:
    data_scaled = np.empty_like(label_data)
    data_scaled[:, 0, :, :] = (label_data[:, 0, :, :] - u_min) / (u_max - u_min)
    data_scaled[:, 1, :, :] = (label_data[:, 1, :, :] - v_min) / (v_max - v_min)

    print(
        f"U component scaled to range: [{data_scaled[:,0, :, :].min()},
↪{data_scaled[:,0, :, :].max()}"]
    )
    print(
        f"V component scaled to range: [{data_scaled[:,1, :, :].min()},
↪{data_scaled[:,1, :, :].max()}"]
    )

    # scale inputs:
    input_scaled = np.empty_like(input_data)
    input_scaled[:, 0, :, :] = (input_data[:, 0, :, :] - u_input_min) / (
        u_input_max - u_input_min
    )

    generateYAML(u_min, u_max, v_min, v_max, u_input_min, u_input_max,
↪yaml_filepath)

    return data_scaled, input_scaled

def unnormalize_data(label_scaled, yaml_filepath):
    with open(yaml_filepath, "r") as f:

```

```

min_max = yaml.safe_load(f)

u_min = min_max["labels"]["u"]["min"]
u_max = min_max["labels"]["u"]["max"]
v_min = min_max["labels"]["v"]["min"]
v_max = min_max["labels"]["v"]["max"]

data_unnormalized = np.empty_like(label_scaled)
data_unnormalized[:, 0, :, :] = label_scaled[:, 0, :, :] * (u_max - u_min)
↪+ u_min
data_unnormalized[:, 1, :, :] = label_scaled[:, 1, :, :] * (v_max - v_min)
↪+ v_min

return data_unnormalized

def split_and_save(label_scaled, input_scaled, train_ratio, test_ratio,
↪save_path):
    data = torch.utils.data.TensorDataset(
        torch.FloatTensor(input_scaled),
        torch.FloatTensor(label_scaled)
    )
    num = len(data)
    train_count = int(num * train_ratio)
    test_count = int(num * test_ratio)
    val_count = num - train_count - test_count
    print(f"Splitting data into Train: {train_count}, Test: {test_count},
↪Validation: {val_count}")

    train_dataset, val_dataset, test_dataset = torch.utils.data.random_split(
        data,
        [train_count, val_count, test_count]
    )
    # convert Subset to TensorDataset for saving
    train_dataset = torch.utils.data.TensorDataset(
        train_dataset.dataset.tensors[0][train_dataset.indices],
        train_dataset.dataset.tensors[1][train_dataset.indices],
    )
    test_dataset = torch.utils.data.TensorDataset(
        test_dataset.dataset.tensors[0][test_dataset.indices],
        test_dataset.dataset.tensors[1][test_dataset.indices],
    )
    val_dataset = torch.utils.data.TensorDataset(
        val_dataset.dataset.tensors[0][val_dataset.indices],
        val_dataset.dataset.tensors[1][val_dataset.indices],
    )

```

```

torch.save(train_dataset, save_path + "train_dataset.pt")
torch.save(test_dataset, save_path + "test_dataset.pt")
torch.save(val_dataset, save_path + "validation_dataset.pt")
print(f"Datasets saved to {save_path}")

def generateYAML(u_min, u_max, v_min, v_max, input_min, input_max,
    ↪yaml_filepath):
    min_max = {
        "inputs": {"u": {"max": float(input_max), "min": float(input_min)}},
        "labels": {
            "u": {"max": float(u_max), "min": float(u_min)},
            "v": {"max": float(v_max), "min": float(v_min)},
        },
    }

    # Save to yaml file
    print(f"Saving min_max.yaml to {yaml_filepath}")
    with open(yaml_filepath, "w") as f:
        yaml.dump(min_max, f, default_flow_style=False)

    print("min_max.yaml file created successfully!")
    print("\nContents:")
    print(yaml.dump(min_max, default_flow_style=False))

```

```

[3]: velocity_range = (0.5, 1.5)
num_samples = 101
train_ratio = 0.8
test_ratio = 0.1
in_data_dir = "../out/"
# in_data_dir = "/kaggle/input/101-lid-driven-cavities/" #Kaggle
# in_data_dir = "/content/" #Google
out_pre = "../out_ml/"
# out_pre = "/kaggle/working/" #Kaggle
# out_pre = "/content/" #Google

label_all, input_all = generate_data(in_data_dir,
    ↪velocities_range=velocity_range, num_samples=num_samples)
plot_sample_data(label_all, input_all, sample_index=0)
label_scaled, input_scaled = normalize_data(label_all, input_all,
    ↪f"{out_pre}min_max.yaml")
data = torch.utils.data.TensorDataset(
    torch.FloatTensor(input_scaled),
    torch.FloatTensor(label_scaled)
)
split_and_save(label_scaled, input_scaled, train_ratio=train_ratio,
    ↪test_ratio=test_ratio, save_path=f"{out_pre}")

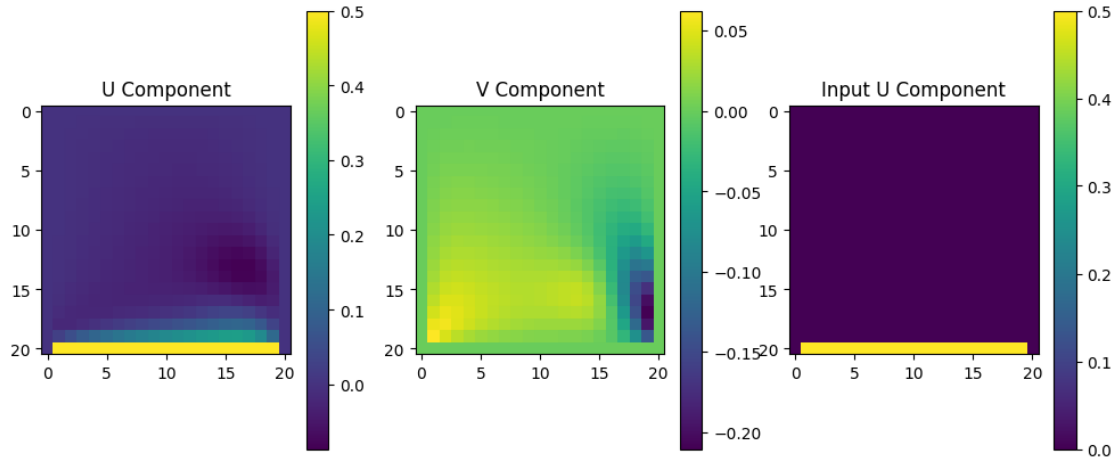
```


Generated 101 samples of data.

Each sample has shape: (2, 21, 21) for labels and (1, 21, 21) for inputs.

Label data shape: (101, 2, 21, 21)

Input data shape: (101, 1, 21, 21)



Input U component original range: [0.0, 1.5]

U component original range: [-0.12409422607456072, 1.4999998720029535]

V component original range: [-0.35372578603195964, 0.09177235098245812]

U component scaled to range: [0.0, 1.0]

V component scaled to range: [0.0, 1.0]

Saving min_max.yaml to ./out_ml/min_max.yaml

min_max.yaml file created successfully!

Contents:

inputs:

u:

max: 1.5

min: 0.0

labels:

u:

max: 1.4999998720029535

min: -0.12409422607456072

v:

max: 0.09177235098245812

min: -0.35372578603195964

Splitting data into Train: 80, Test: 10, Validation: 11

Datasets saved to ./out_ml/

```
[ ]: from mymodel import init_my_model # assuming mymodel.py is in the same
     ↪ directory
```

```

def train_model(model, optimizer, criterion, device, epochs, train_loader,
    ↪ val_loader, scaler, scheduler):
    """
    Train the model

    :param model: Model to be trained
    :param optimizer: Optimizer for training
    :param criterion: Loss function
    :param device: Device to run the training on
    :param epochs: Number of epochs to train
    :return: train_losses, val_losses
    """
    train_losses = []
    val_losses = []
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        val_loss = 0.0

        for batch_input, batch_label in train_loader:
            inputs, labels = batch_input.to(device), batch_label.to(device)
            optimizer.zero_grad()
            y_pred = model(inputs)
            loss = criterion(y_pred, labels)

            # SCALER
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            #optimizer.step()
            running_loss += loss.item() * inputs.size(0)

        running_loss /= len(train_loader.dataset)

        model.eval()
        with torch.no_grad():
            for val_input, val_label in val_loader:
                ↪ val_input.to(device)
                y_val_pred = model(val_input)
                val_loss += criterion(y_val_pred, val_label)*val_input.size(0)
            val_loss /= len(val_loader.dataset)

        #SCHEDULER
        if scheduler is not None:

```

```

        scheduler.step()

    train_losses.append(running_loss)
    val_losses.append(val_loss.item())

    if (epoch + 1) % 200 == 0:
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch {epoch+1}/{epochs} | Train: {running_loss:.4e} | Val: {val_loss:.4e} | LR: {current_lr:.2e}")

    print("Training completed!")
    return train_losses, val_losses

# PARAM is defined in init my model: torch.nn.SiLU()
mymodel = init_my_model()
print(mymodel)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"running on device {device}")
mymodel.to(device)

# PARAMS
epochs = 4000
lr = 5e-4
batch_size = 16
criterion = torch.nn.MSELoss()
optimizer = torch.optim.AdamW(mymodel.parameters(), lr)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
scaler = torch.amp.GradScaler()

train_dataset = load_data(f"{out_pre}train_dataset.pt")
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True # SHUFFLE SHUFFLE
)

print(f"min, max of training inputs and labels: {train_dataset.tensors[0].min()}, {train_dataset.tensors[0].max()}, {train_dataset.tensors[1].min()}, {train_dataset.tensors[1].max()}")

test_dataset = load_data(f"{out_pre}test_dataset.pt")
test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=batch_size, shuffle=False # SHUFFLE SHUFFLE
)

print(f"min, max of test inputs and labels: {test_dataset.tensors[0].min()}, {test_dataset.tensors[0].max()}, {test_dataset.tensors[1].min()}, {test_dataset.tensors[1].max()}")

```

```

val_dataset = load_data(f"{out_pre}validation_dataset.pt")
val_loader = torch.utils.data.DataLoader(
    val_dataset, batch_size=batch_size, shuffle=False # SHUFFLE SHUFFLE SHUFFLE
)
print(f"min, max of val inputs and labels: {val_dataset.tensors[0].min()}, \u
    \u2192{val_dataset.tensors[0].max()}, {val_dataset.tensors[1].min()}, {val_dataset.
    \u2192tensors[1].max()}")

start = time.time()
train_losses, val_losses = train_model(
    mymodel, optimizer, criterion, device, epochs, train_loader, val_loader, \u
    \u2192scaler, scheduler
)
print(f"Training time: {time.time() - start} seconds")
np.save(f"{out_pre}train_losses.npy", np.array(train_losses))
np.save(f"{out_pre}val_losses.npy", np.array(val_losses))
save_params(mymodel, f"{out_pre}model.pt")
plot_loss(train_losses, val_losses)

```

```

MyModel(
    (conv1): Conv2d(1, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af1): SiLU()
    (conv2): Conv2d(16, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af2): SiLU()
    (conv3): Conv2d(16, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af3): SiLU()
    (conv4): Conv2d(16, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af4): SiLU()
    (conv5): Conv2d(16, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af5): SiLU()
    (conv6): Conv2d(16, 16, kernel_size=(7, 7), stride=(1, 1), padding=same)
    (af6): SiLU()
    (conv7): Conv2d(16, 2, kernel_size=(7, 7), stride=(1, 1), padding=same)
)
running on device cpu

/tmp/ipykernel_67877/31710212.py:73: UserWarning: torch.cuda.amp.GradScaler is
enabled, but CUDA is not available. Disabling.
    scaler = torch.amp.GradScaler()

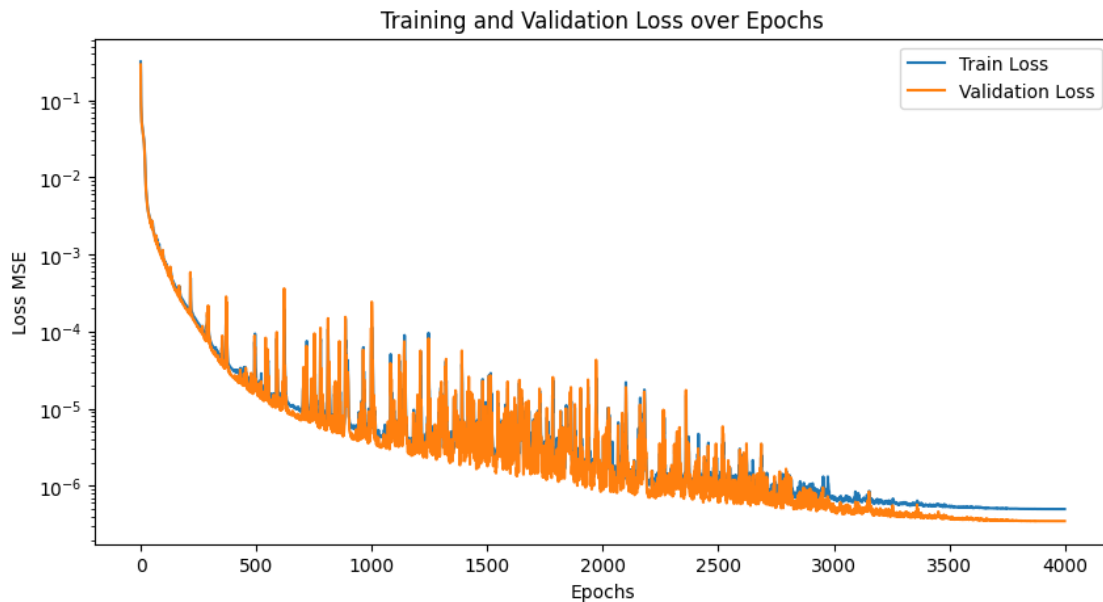
Loading dataset from ./out_ml/train_dataset.pt
shape of loaded dataset: Input: torch.Size([80, 1, 21, 21]) and labels:
torch.Size([80, 2, 21, 21])
min, max of training inputs and labels: 0.0, 1.0, 0.0, 1.0
Loading dataset from ./out_ml/test_dataset.pt
shape of loaded dataset: Input: torch.Size([10, 1, 21, 21]) and labels:
torch.Size([10, 2, 21, 21])
min, max of test inputs and labels: 0.0, 0.99333333396911621,

```

```

0.0002646878419909626, 0.9994076490402222
Loading dataset from ./out_ml/validation_dataset.pt
shape of loaded dataset: Input: torch.Size([11, 1, 21, 21]) and labels:
torch.Size([11, 2, 21, 21])
min, max of val inputs and labels: 0.0, 0.9266666769981384,
0.0010648267343640327, 0.994961678981781
Epoch 200/4000 | Train: 2.1531e-04 | Val: 1.9201e-04 | LR: 4.97e-04
Epoch 400/4000 | Train: 3.1735e-05 | Val: 2.5616e-05 | LR: 4.88e-04
Epoch 600/4000 | Train: 1.6249e-05 | Val: 1.1663e-05 | LR: 4.73e-04
Epoch 800/4000 | Train: 7.2492e-06 | Val: 6.0847e-06 | LR: 4.52e-04
Epoch 1000/4000 | Train: 7.7497e-05 | Val: 1.2481e-05 | LR: 4.27e-04
Epoch 1200/4000 | Train: 1.1246e-05 | Val: 3.4404e-06 | LR: 3.97e-04
Epoch 1400/4000 | Train: 4.5276e-06 | Val: 3.3832e-06 | LR: 3.63e-04
Epoch 1600/4000 | Train: 2.1865e-06 | Val: 1.7125e-06 | LR: 3.27e-04
Epoch 1800/4000 | Train: 2.0763e-06 | Val: 1.4177e-06 | LR: 2.89e-04
Epoch 2000/4000 | Train: 1.4070e-06 | Val: 1.0413e-06 | LR: 2.50e-04
Epoch 2200/4000 | Train: 1.0578e-06 | Val: 7.8123e-07 | LR: 2.11e-04
Epoch 2400/4000 | Train: 1.3783e-06 | Val: 8.4954e-07 | LR: 1.73e-04
Epoch 2600/4000 | Train: 1.4421e-06 | Val: 8.7746e-07 | LR: 1.37e-04
Epoch 2800/4000 | Train: 1.3060e-06 | Val: 1.2492e-06 | LR: 1.03e-04
Epoch 3000/4000 | Train: 6.9849e-07 | Val: 5.0331e-07 | LR: 7.32e-05
Epoch 3200/4000 | Train: 6.0372e-07 | Val: 4.3399e-07 | LR: 4.77e-05
Epoch 3400/4000 | Train: 5.5972e-07 | Val: 4.0161e-07 | LR: 2.72e-05
Epoch 3600/4000 | Train: 5.2967e-07 | Val: 3.7839e-07 | LR: 1.22e-05
Epoch 3800/4000 | Train: 5.0592e-07 | Val: 3.5342e-07 | LR: 3.08e-06
Epoch 4000/4000 | Train: 5.0173e-07 | Val: 3.5057e-07 | LR: 0.00e+00
Training completed!
Training time: 715.0149943828583 seconds
Saving model to ./out_ml/model.pt

```



```
[5]: test_loss, y_pred_all = evaluate_model(mymodel, criterion, device, test_dataset)
test_loss2, y_pred_all2 = evaluate_model(mymodel, criterion, device, data)
plot_test_results(test_dataset, y_pred_all, num_samples=3)
np.save(f"{out_pre}train_losses.npy", np.array(train_losses))
np.save(f"{out_pre}val_losses.npy", np.array(val_losses))
```

Test Loss: 1.0200851647823583e-06

Test Loss: 5.36592324351659e-07

